Adding circuits and Plexers

Original slides by: Jarrett Billingsley
Modified with bits from: Bruce Childers, David Wilkinson

CS 0447
Introduction to
Computer Programming

Luís Oliveira

Fall 2020

Class announcements

- none

Can I add with transistors?

Solving a problem

- Let's say we want to create a circuit to add two bits.
  - How do we do that?

- In 3 very simple steps:
  - Create a truth table that accurately represents the problem
  - Interpret the truth table into a logic function
  - Translate the function into a circuit

The Tables of Truth

- let's try to come up with a **truth table** for adding two bits
- each column will hold **1 bit**

let's name the inputs A and B

for the input values, we **count up in binary**

A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

let's name the outputs **C** and **S**, for Carry and Sum

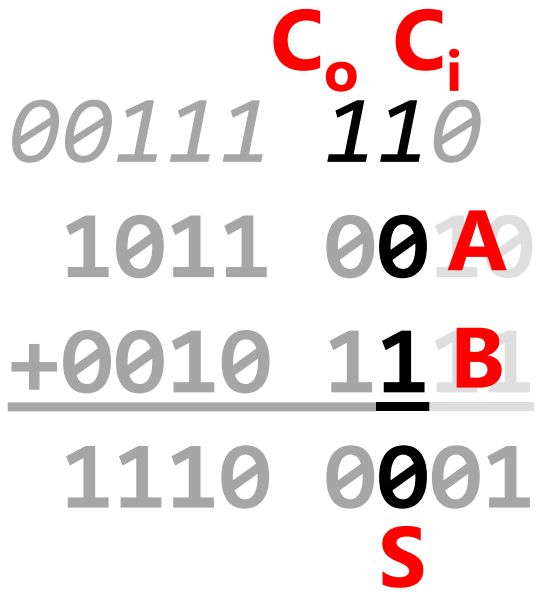
now let's fill in the output values

hey, this C column looks familiar... so does the S column

great! But this is wrong.

Half-truth tables

- what we just made was a **half-adder**
- it has a carry *output* but not a carry *input*
  - (which might be useful for the lowest bit)
- to make a **full adder,** we need **3 input bits**



C_i	A	B	C_o	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

The logic of it all

- it looks a little messy, but it kinda makes sense if you think of it like this:
  - it **counts how many input bits are "1"**
  - C_o and S are a **2-bit number!**
- if we look at the outputs in isolation:
  - S is 1 if we have an **odd number of "1s"**
  - Co is 1 if we have **2 or 3 "1s"**
- it's a little weird, but we can build this out of AND, OR, and XOR gates

C_i	A	B	C_o	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Let's build the adder Co circuit

A	B	C_i	C_o	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$C_{out} = \bar{A}BC_{in} + A\bar{B}C_{in} + AB\overline{C_{in}} + ABC_{in}$$

$\bar{A}BC_{in}$

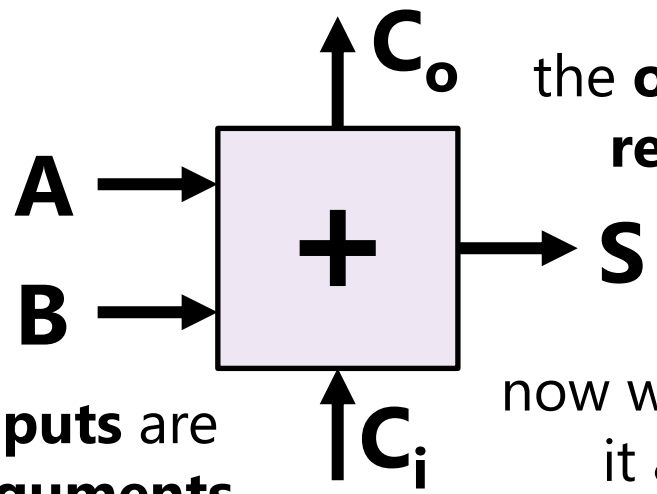
$A\bar{B}C_{in}$

$AB\overline{C_{in}}$

ABC_{in}

Sweeping that under the rug...

- in programming, we use **functions** to be able to reuse code
- in hardware, we can group gates into a **component**
- here's the symbol for a **one-bit full adder**



the **outputs** are like **return values**

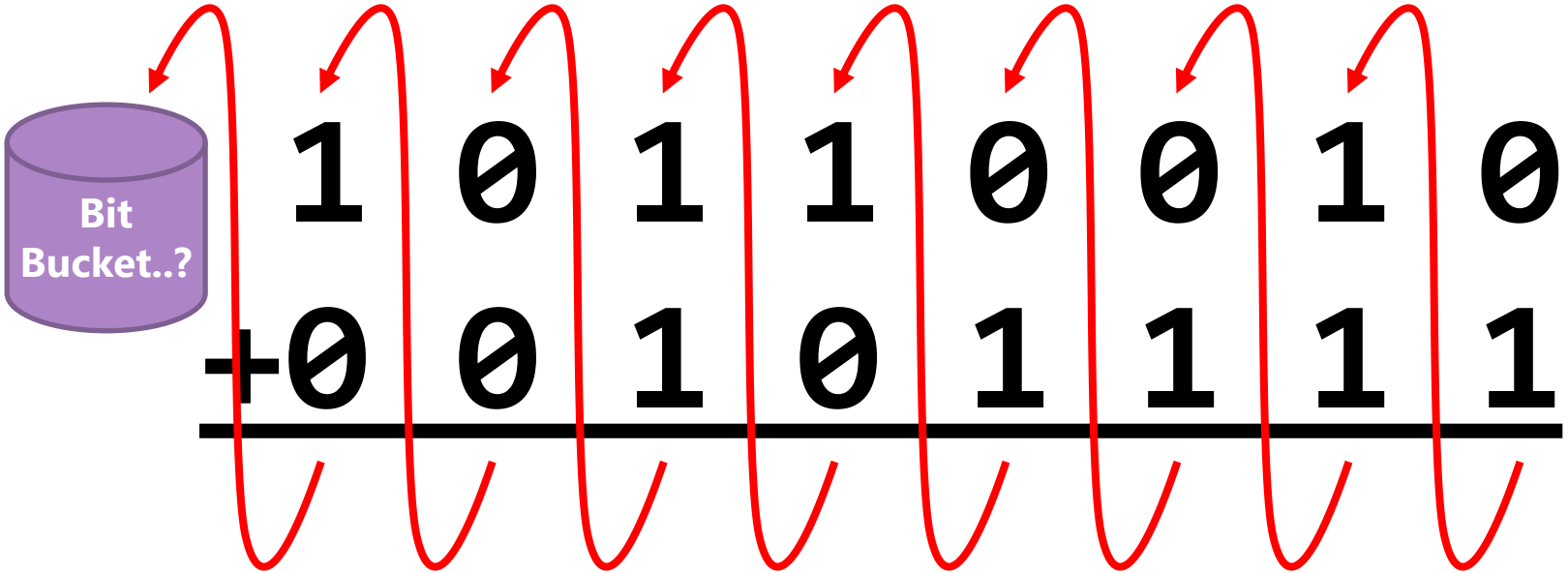
the **inputs** are like **arguments**

now we don't have to care *how* it adds, just that it *does*

Adding longer numbers

Where do the carries go?

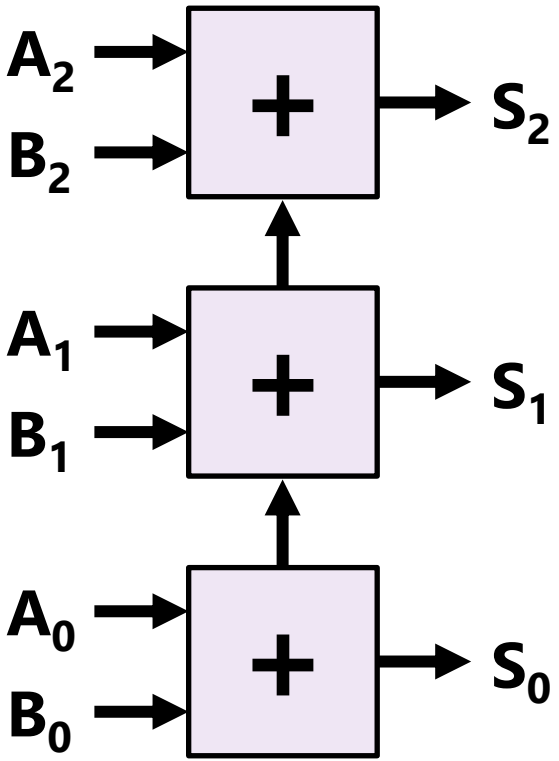
- when you add one place, you might get a **carry out**
- that becomes the **carry in** for the next higher place



Ripple Carry Adder

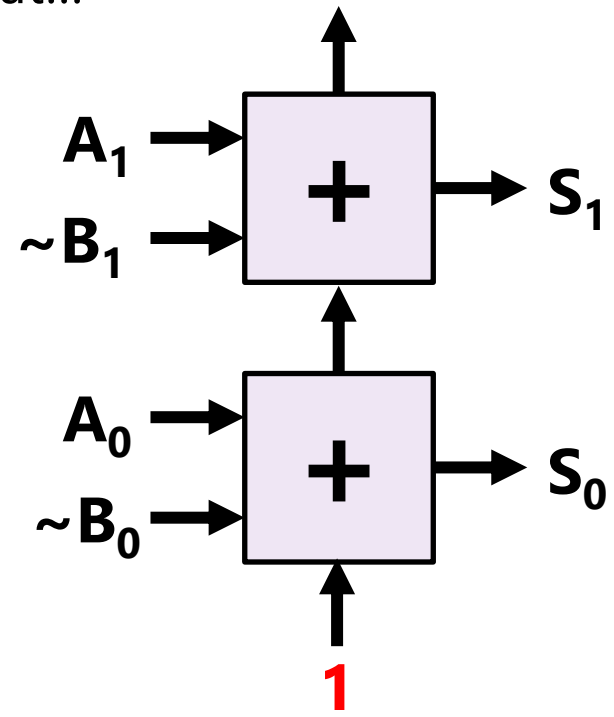
- if we want to add two **three-bit numbers,** we'll need three **one-bit** adders
- we chain the carries from each place to the **next higher place,** like we do on paper
- we have to split the numbers up like so:

	A_2	A_1	A_0
+	B_2	B_1	B_0
	S_2	S_1	S_0



Flip side

- We *could* come up with a separate subtraction circuit, but…
- Since algebra tells us that **x - y = x + (-y)**
  - Negation meaning **flip the bits** and **add 1**
- Flipping the bits uses NOT gates
- How do we add 1 without any extra circuitry?
  - we use a *full adder for the LSB,* and when we're subtracting, set the "**carry in**" to 1

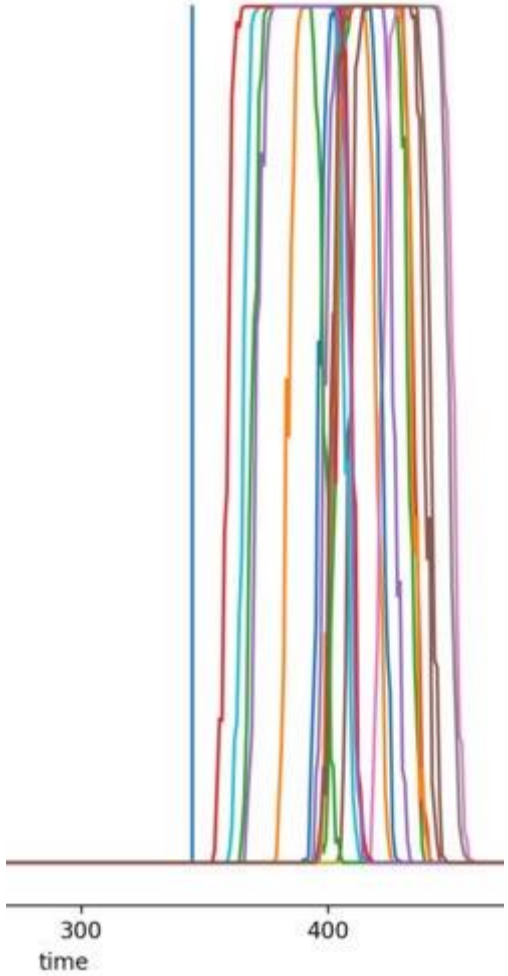


What makes a good word size?

- can you think of an example of…
  - 100 of something?
  - a million of something? One thousand million?
  - One billion? ***more?***
- $2^8 = 256$, $2^{16} \cong 65{,}000$, $2^{32} \cong 4000$ million, $2^{64} \cong$ lots-of-a-lot
- for a given word size, **all the circuitry** has to be built to support it
  - 64 1-bit adders
  - 128 wires going in
  - 64 wires coming out

Gate Delay

- electrical signals **can't move** infinitely fast
- transistors **can't turn on and off** infinitely fast
- since each digit must wait for the next smaller digit to compute its carry…
  - ripple carry is **linear in the number of digits**
- this is a diagram of how the outputs of a 16-bit ripple carry adder change over time
  - it's measured in *picoseconds!* so ~100ps total
- but if we went to 32 bits, it'd take 200ps
  - and 64 bits, 400ps…
- there *are* more efficient ways of adding

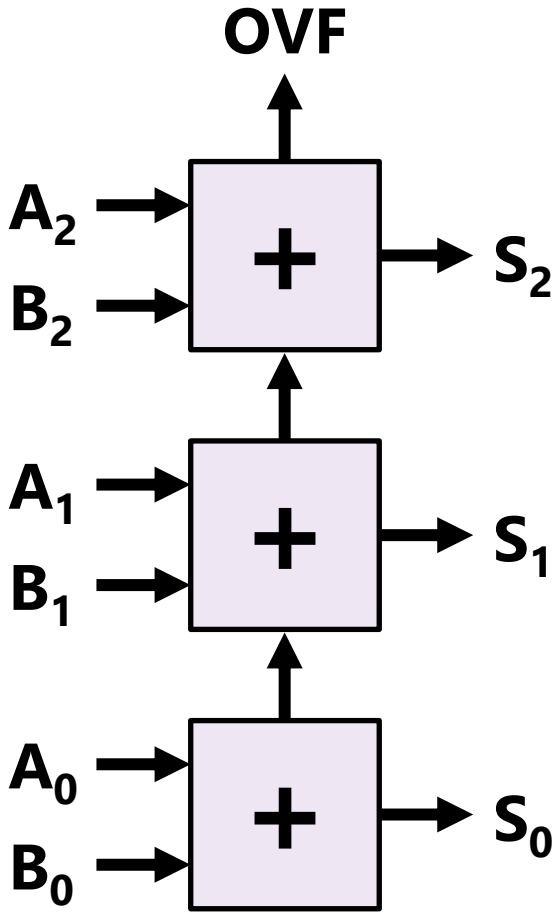


(courtesy of Kate Temkin)

What about overflow?

- For *unsigned* addition, it's easy
  - For an n-bit adder:
    - just look at the **C_o** of the **MSB**
    - if it's 1, **it's an overflow.**
  - what about subtraction?

- For signed, is a bit strange
  - Compare the last 2 carry bits
  - If they are different
    - Then there is overflow



But why?

- When does signed addition overflow?
  - If:
    - Both addends have the same sign
    - The result has a different sign
- How can we detect that?
  - Looking at the last bit!
- Where is the overflow?

The last bit:

C_i	A	B	C_o	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

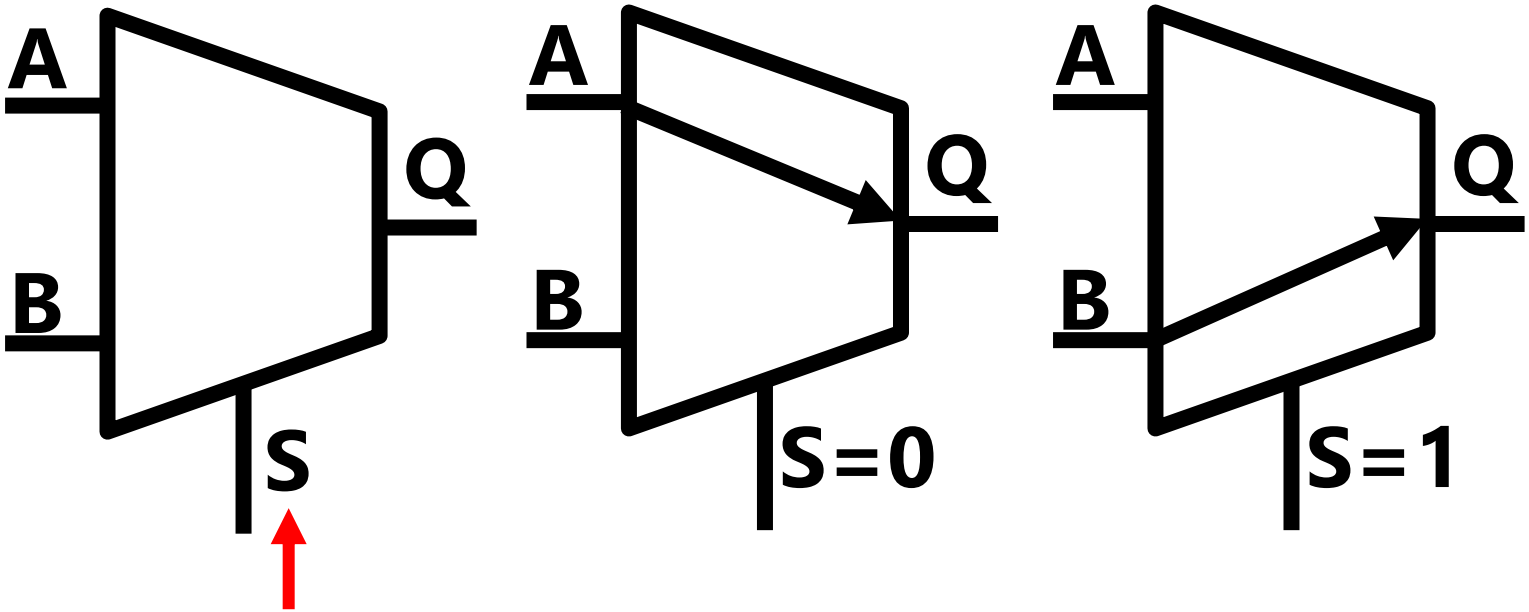
There is overflow in signed addition if:

On the last bit, the carry-in and carry-out have different bit values.

Muxes and demuxes, encoders and decoders

Hardware that makes decisions

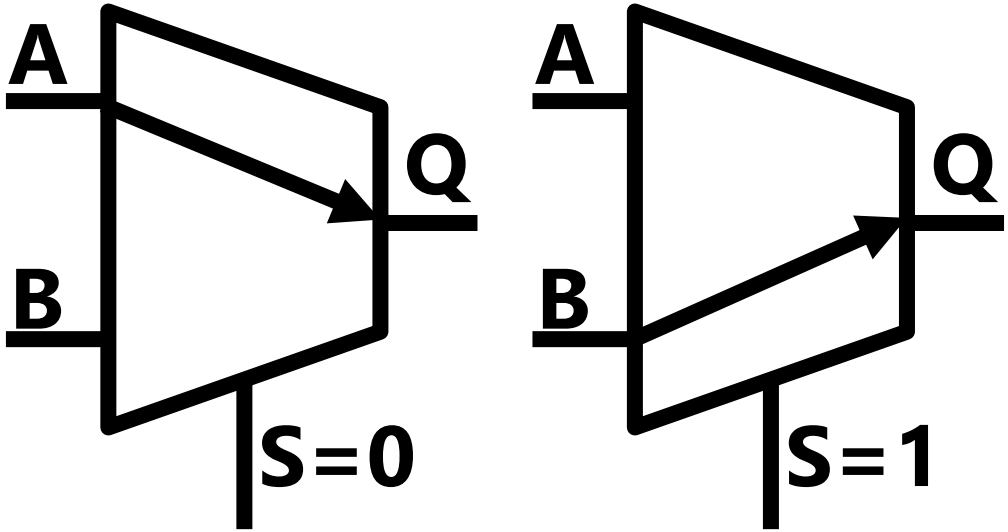
- a **multiplexer** (**mux**) outputs one of its inputs based on a **select.**



This is the select input.

Multiplexer truth table

- let's make a truth table for a **two-input 1-bit multiplexer.**



S	A	B	Q
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Doing everything and throwing most of it away

- I want a circuit that does this:

```
if(select == 1)
    output = A - B
else
    output = A + B
```

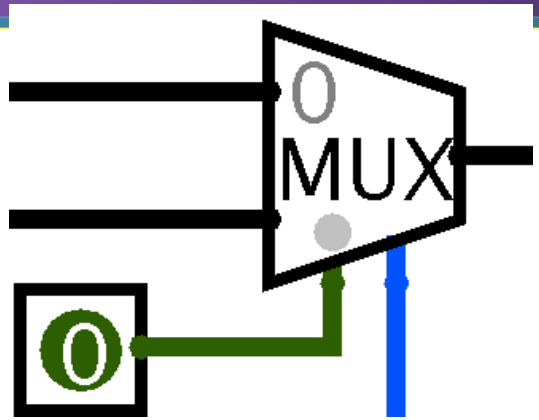
- let's see what that looks like
- a mux is like a **hardware if-else statement**
- but unlike in software...
  - the "condition" comes at the "end" (the output)
  - instead of doing one *or* the other, we **do both, choose the one that we care about, and ignore the rest!**

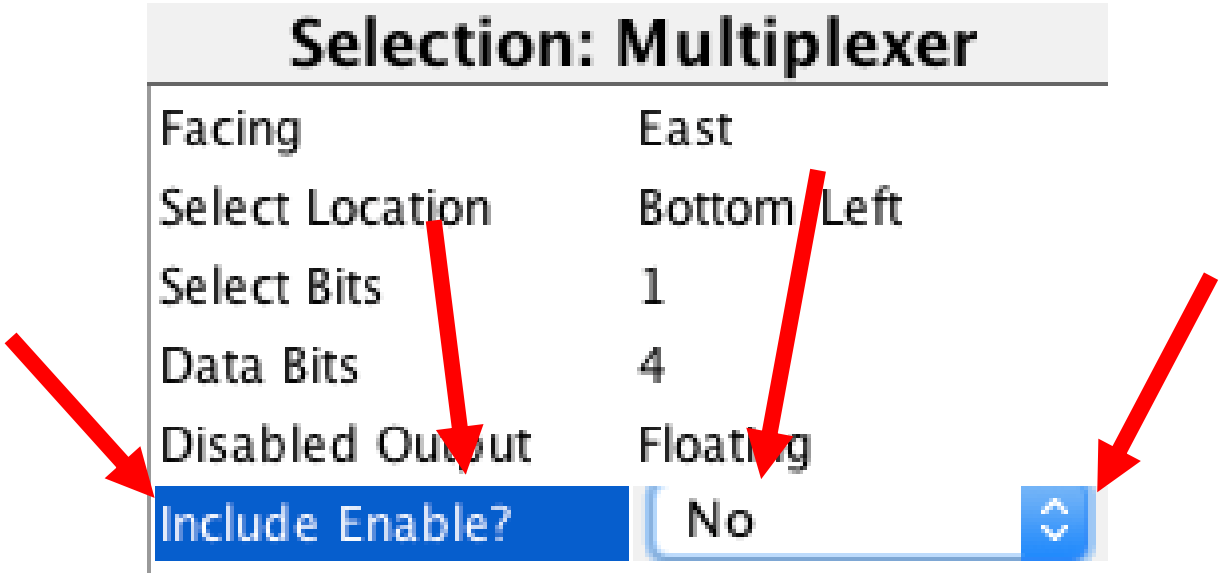
Amusing muxes

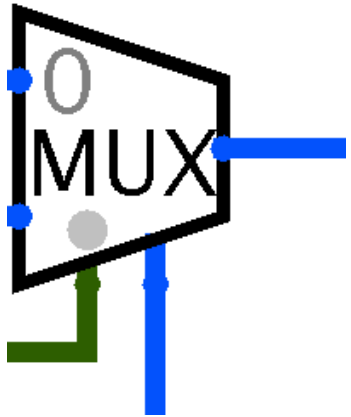
- Let's go to Logisim!

What's that enable input?

- if you don't understand tristate buses or high impedance states, **do not turn on the *enable* input.**
- if you ever see **blue wires**, **you are in weird, confusing territory.**
- if you know this stuff, fine, but otherwise…

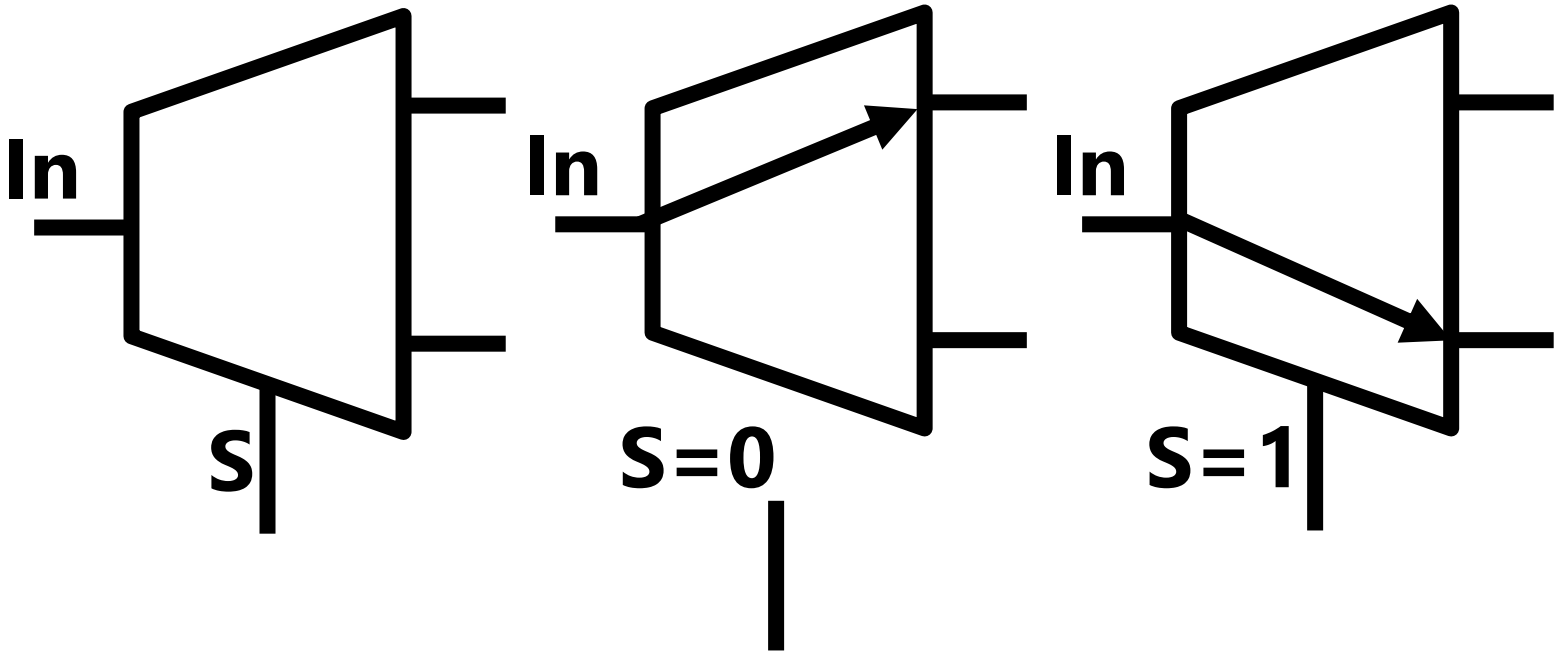






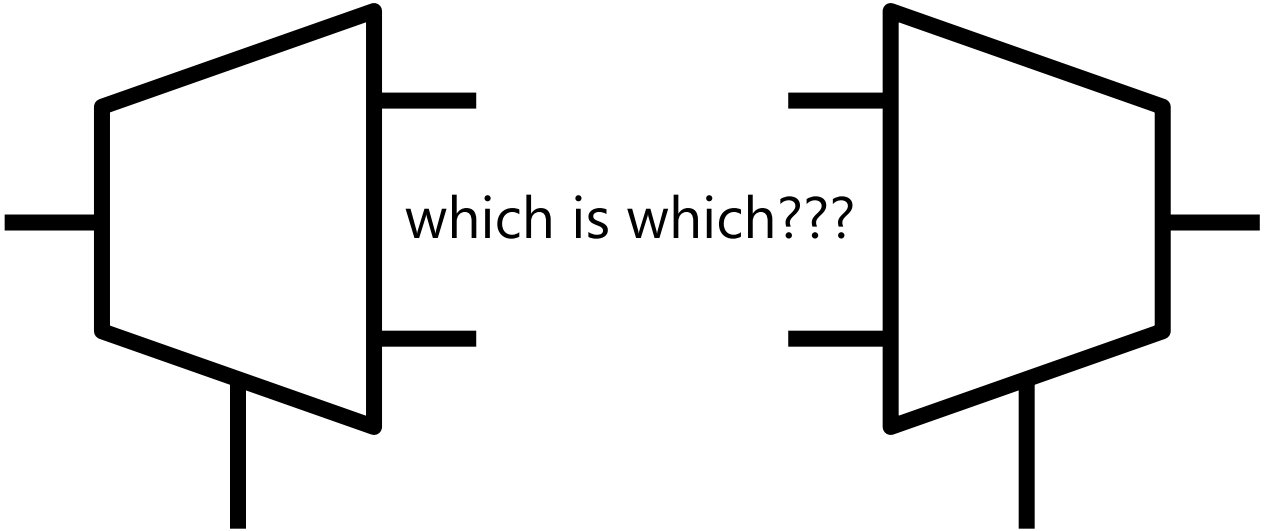
Demultipliexers

- a **demux** does the opposite: it sends its input to one of its outputs
- **the rest of the outputs are 0s**



Looking in a mirror

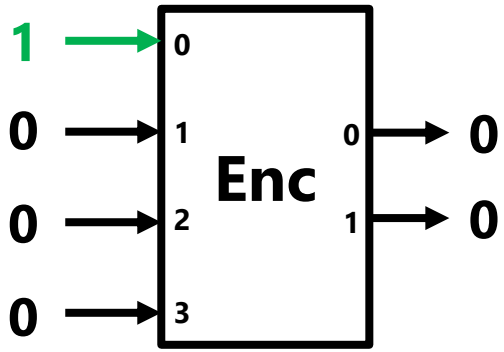
- it can be confusing if all you see is this:

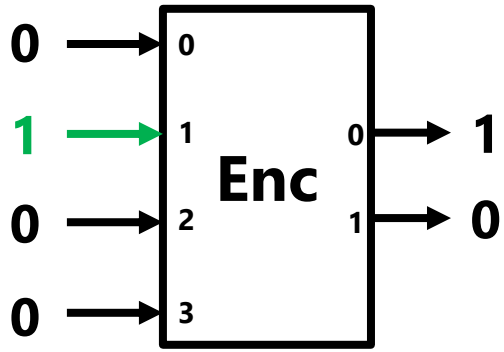


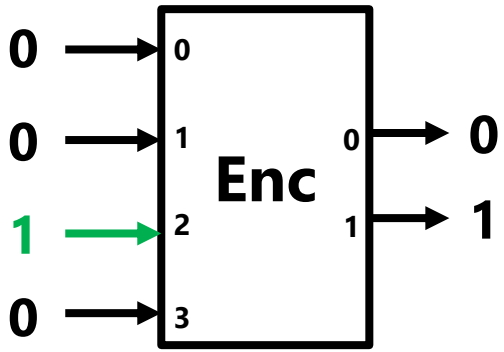
Logisim distinguishes these with names
I'll do it with arrows

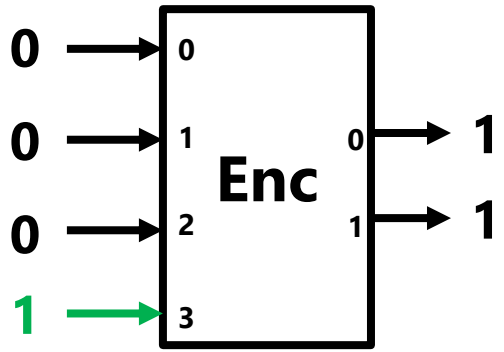
Encoders

- They **encode** 2^n inputs into n outputs. Specifically…
- you give it several 1-bit inputs, and it tells you **which one is 1.**







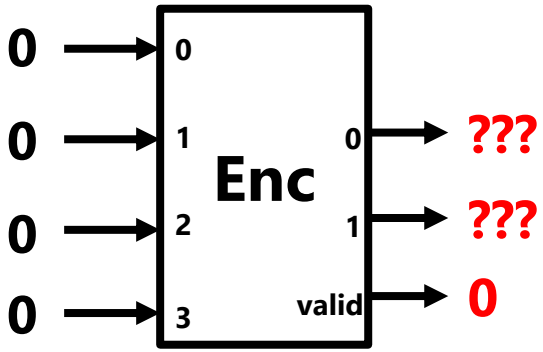


I_0	I_1	I_2	I_3	Out
1	0	0	0	00
0	1	0	0	01
0	0	1	0	10
0	0	0	1	11

Encoder issues

- That table seems VERY incomplete!!!
  - What about the other entries???

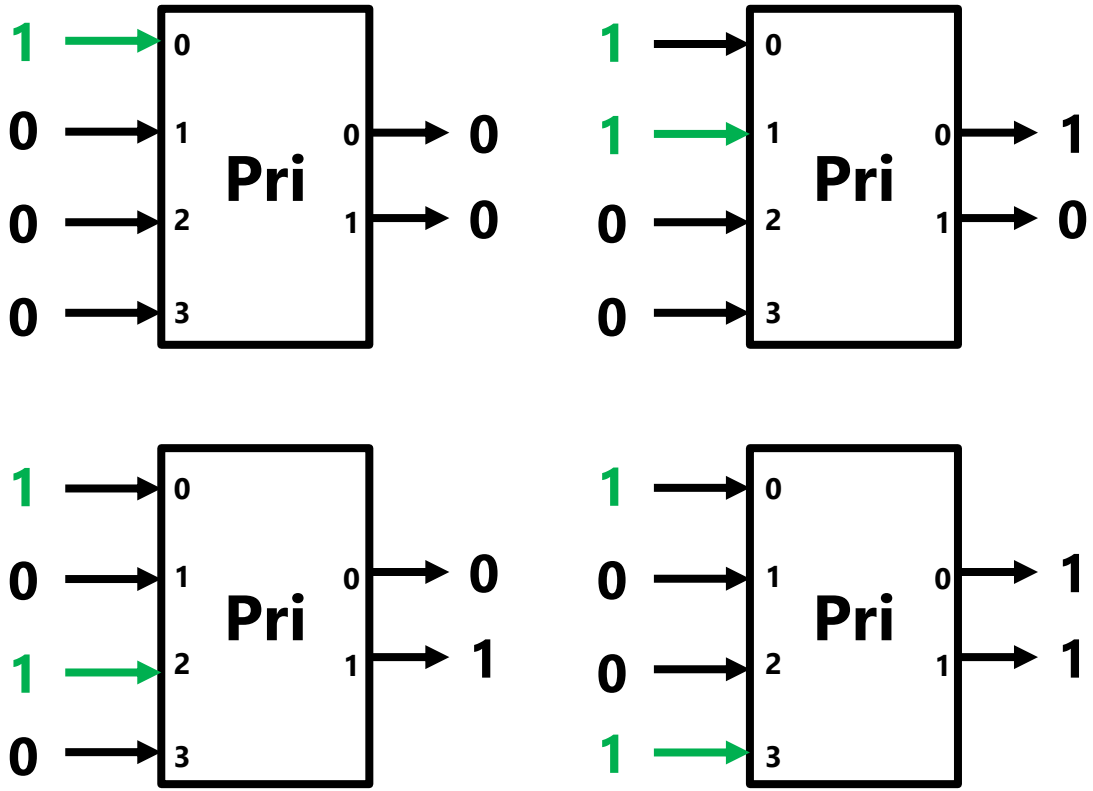
The output is not valid!



I_0	I_1	I_2	I_3	Out	Valid
0	0	0	0	??	0
0	1	1	0	??	0
1	1	1	0	??	0
1	1	1	1	??	0
…	…	…	…	…	…

Enter: Priority Encoders

- In a **priority encoder** … you give it several 1-bit inputs, and it tells you the **highest input with a 1.**



I_0	I_1	I_2	I_3	Out
1	0	0	0	00
X	1	0	0	01
X	X	1	0	10
X	X	X	1	11

What are these?

idc

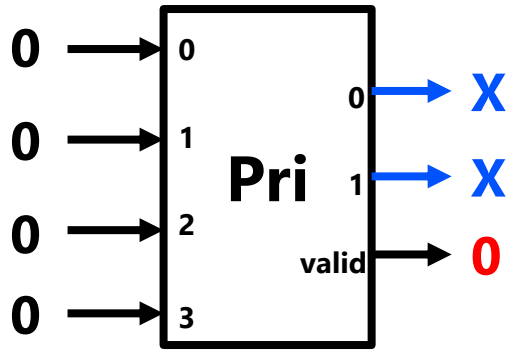
- we don't even *care* about the *that* input
- we can put **X** in the inputs we don't care about
- we call these **don't cares**
  - yep, really
- what these mean is:
  - when we make this into a boolean function, we can **ignore** those inputs
    - we won't even need to write em

I_0	I_1	I_2	I_3	Out
1	0	0	0	00
X	1	0	0	01
X	X	1	0	10
X	X	X	1	11

Still…

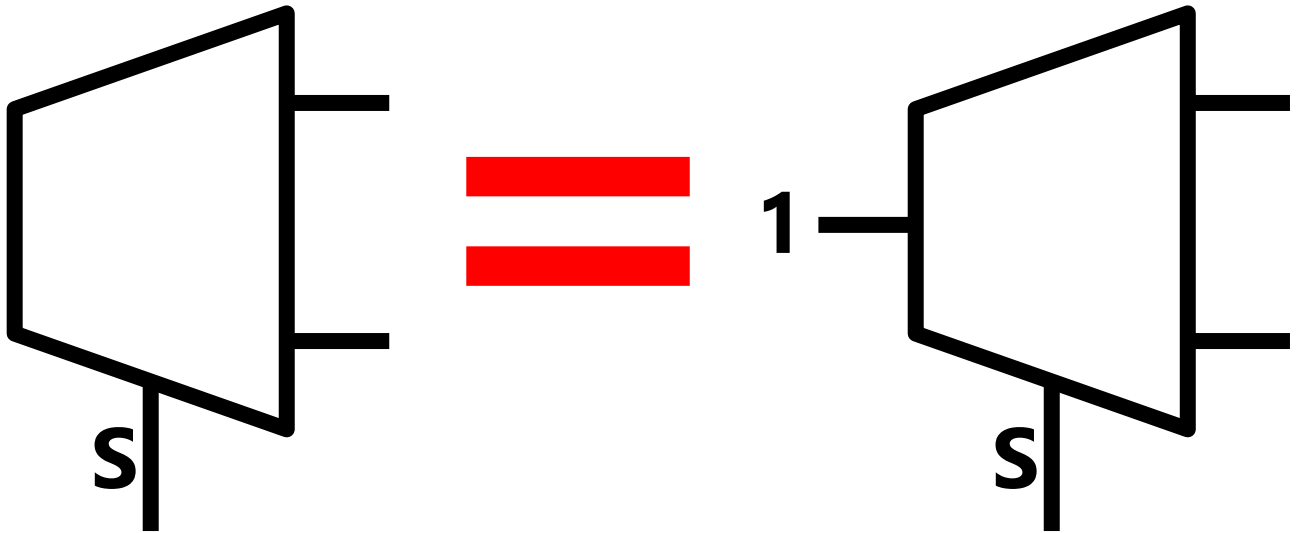
- All **zeros** is still an invalid input :(

if *none* of the inputs is 1,
then logisim gives you X…
These **are not** don't cares :')



Decoders

- a **decoder** is like a 1-bit demux whose input is always 1
- It does pretty much the opposite of an encoder ☺



exactly one output is 1,
and the rest are 0s

Uses for encoders, decoders and demuxes

- uhhhhhhhhhh
  - Ummmmmmmmm... for now ...
    - unless you're using tristate (blue) wires, they're not too useful...
- most of the time, you don't have to "direct" a signal to a location
  - instead, you **hook up the inputs to everything that needs them**
- we'll use them more when we get to **sequential logic**

Combinational vs Sequential

- **combinational logic:** the outputs of a circuit depend entirely on their ***current*** inputs
  - AND, OR, NOT, XOR gates
  - adders
  - muxes, demuxes, encoders, and decoders
- **sequential logic** is coming up soon
  - the outputs can depend on the **current *and previous*** inputs
  - it *remembers*
- logic minimization techniques only work on **combinational** logic!
  - …or combinational *pieces* of a larger sequential circuit